

Министерство науки и высшего образования Российской Федерации
Московский физико-технический институт (государственный университет)

Физтех-школа прикладной математики и информатики
Кафедра дискретной математики

Выпускная квалификационная работа бакалавра

Постквантовая криптография на решётках.

Автор:

Студент Б05-228 группы
Савватеев Михаил Алексеевич

Научный руководитель:

Доктор физико-математических наук
Фролов Алексей Андреевич



Москва 2026

Аннотация

Постквантовая криптография на решётках.

Савватеев Михаил Алексеевич

Большинство современных криптографических протоколов являются уязвимыми к атаке на квантовом компьютере. Для исправления ситуации разрабатываются новые протоколы, многие из которых основаны на теории решёток. Вводится задача решения зашумлённой системы уравнений **LWE** и её аналоги над кольцами многочленов: **RLWE** и **MLWE**. На их основе получаются уже конкретные устойчивые относительно квантового взломщика криптосистемы. В данной работе процесс шифрования рассматривается, как передача данных по зашумлённому **RLWE**-каналу. Используются Error Correction Codes для исправления ошибок при декодировании, исследуется зависимость их параметров от используемого размера алфавита. Далее подбирается конфигурация для достижения максимальной скорости передачи и сравниваются результаты при использовании метрик Хэмминга и Ли соответственно.

Содержание

1	Введение	4
2	Обозначения	6
3	Постановка задачи	8
4	Обзор существующих решений	11
5	Исследование и построение решения задачи	13
5.1	Свойства маппинга	13
5.2	Вычисление DFR	13
5.3	Границы на скорость кода	14
6	Описание практической части	16
6.1	Вычисление шума	16
6.2	Вероятности ошибок	17
6.3	Подсчёт t и R	18
7	Численные результаты	19
8	Заключение	22
	Приложение	27

1 Введение

Большая часть современных криптографических протоколов базируются на сложности задач факторизации натурального числа и дискретного логарифмирования элемента в заданной абелевой группе.

Таковыми являются, например, широко известная криптосистема RSA (*Rivest–Shamir–Adleman*) [1], а также современные российские ГОСТы (*государственные стандарты*) для цифровой подписи документов [2] и получения двумя сторонами общего секретного ключа [3].

При этом в 1994 - 1997 годах Шор разработал алгоритм на квантовом компьютере, который решает обе эти задачи в $\mathbb{N}$ за полиномиальное от длины числа время [4]. В дальнейшем алгоритм улучшался и модифицировался, в том числе, в 2003 году была опубликована и подробно описана его версия на эллиптических кривых [5].

Сейчас многими компаниями и государствами ведутся исследования по построению мощных стабильных квантовых компьютеров. В случае успеха оных все вышеупомянутые протоколы окажутся уязвимыми.

В связи с этим, начиная с 2016 - 2017 годов NIST (*National Institute of Standards and Technology в США*) проводит конкурс по отбору лучших протоколов постквантовой (*устойчивой к атакам в том числе на квантовом компьютере*) криптографии [6].

Немалая доля из них основана на решёточной криптографии вообще и **LWE**-(*Learning With Errors*)-problem в частности [7]. Одним из главных преимуществ этих криптосистем является наличие **ACH** (*Average-Case*), а не **WCH** (*Worst-Case*) сложности (*Hardness*) [8].

Неформально говоря, это означает, что трудными для взлома являются случайные параметры протокола, а не худшие из всех возможных. В реальных криптосистемах данное свойство очень полезно, поскольку генерация рандомных бит при шифровании необходима.

К сожалению, протоколы, основанные **LWE**-problem, хотя являются простыми и понятными, требуют довольно много вычислений. Поэтому с

целью оптимизации в статье [9] вводится **RLWE**-(*Ring LWE*)-problem. Это модификация **LWE** в многочленах над $\mathbb{Z}_q$ с ускоренными вычислениями за счёт **NTT** (*Number Theoretic Transform*) [10].

Более подробно все эти и некоторые дальнейшие концепции описаны в вышедшем в 2022 году обзоре [11]. В моей же работе я буду использовать и улучшать результаты следующей статьи [12].

В ней процесс **RLWE**-шифрования рассматривается, как передача данных по зашумлённому **RLWE**-каналу. Исследуются свойства этого канала, описывается, как увеличить скорость передачи и при этом снизить **DFR** (*Decryption Failure Probability/Rate*) при расшифровке сообщения.

В частности, для уменьшения **DFR** при фиксированном **CFR** (*Coefficient Failure Rate*) используются **ECC** (*Error Correcting Codes*) [13]. Эта конструкция позволяет исправлять вплоть до t покоординатных ошибок при передаче вектора-сообщения фиксированной длины n (здесь n, t — некоторые выбираемые исследователем параметры).

При этом измерять кол-во произошедших ошибок можно по-разному. В моей работе я пробую разные способы (метрика Хэмминга [13], метрика Ли [14]) и вычисляю, как изменяется от этого скорость **RLWE**-канала.

Как и в статье [12], я варьирую размер алфавита Q , которому принадлежат символы сообщения, и наблюдаю, как изменяются от этого оптимальные параметры канала.

Для этого я выбираю несколько конкретных криптосистем (**LAC 256** [15], **NewHope 1024** [16]), целевое **DFR** и исследую их. В дальнейшем реализованный код и применённые методы можно обобщить также на другие криптосистемы с иными размерами алфавита и метриками.

2 Обозначения

Под $\mathbb{Z}_m$ будем подразумевать кольцо $\mathbb{Z}/m\mathbb{Z}$ с обычными представителями $\{0, \dots, m-1\}$. Ими же зададим стандартное вложение $\mathbb{Z}_m \hookrightarrow \mathbb{Z}$, сравнение из которого будем использовать, говоря $x < y$ для $x, y \in \mathbb{Z}_m$.

Пусть $n \in \mathbb{N}$. Обозначим за $\mathcal{R}^n$ кольцо многочленов $\mathbb{Z}[x]/(x^n + 1)$. Вспомним, что если n — степень 2, то $\mathcal{R}^n$ является полем, ведь $x^n + 1$ будет многочленом деления круга $\implies$ неприводимым над $\mathbb{Z}$. Если дополнительно число q — простое, примем за $\mathcal{R}_q^n$ кольцо $\mathbb{Z}_q[x]/(x^n + 1)$.

Элементы $\mathcal{R}_q^n$ можно считать многочленами степени $n-1$ с n коэффициентами из $\mathbb{Z}_q$. Будем писать их жирным шрифтом $\mathbf{a}$. При этом их i -ый коэффициент будем обозначать a_i . Если все коэффициенты независимо генерируются из распределения P или равномерно из мн-ва S , будем обозначать это $\mathbf{a} \leftarrow P$ или $\mathbf{a} \leftarrow S$ соответственно.

Сумма и произведение естественным образом наследуются в $\mathcal{R}_q^n$ из $\mathbb{Z}_q[x]$, причём из-за выполнения $x^n = -1$ получаем равенство:

$$(\mathbf{ab})_k = \sum_{i=0}^k a_i b_{k-i} - \sum_{i=k+1}^{n-1} a_i b_{n+k-i}$$

Для распределения P будем обозначать вероятность исхода x , как $P[x]$. Если при этом P принимает значения в $\{0, \dots, d-1\}$, ему естественным образом сопоставляется многочлен:

$$\hat{P}(z) = \sum_{i=0}^{d-1} P[i] z^i$$

Для события A примем за $\Pr(A)$ его вероятность, а за $I\{A\}$ — индикатор его выполнения.

Станем обозначать биномиальное распределение с n попытками и вероятностью успеха p , как $\mathcal{B}(n, p)$; оно принимает значения $0 \leq x \leq n$. Если $\mu, \nu \sim \mathcal{B}(k, \frac{1}{2})$ — независимые случайные величины, распределение разности $\mu - \nu$ обозначим за χ_k (*centered binomial distribution*), соответственно, его значения будут $-k \leq x \leq k$.

Свёртку в $\mathbb{Z}_q$ распределений P и R будем писать, как $P * R$. Свёртку же распределения P с собой $n - 1$ раз изобразим $P^{\otimes n}$.

Под алфавитом размера Q будем подразумевать $\Sigma_Q = \{0, \dots, Q - 1\}$. Введём на Σ_Q норму Хэмминга $|s|_H$ и норму Ли $|s|_L$, как:

$$|s|_H = I\{s \neq 0\}; \quad |s|_L = \min(s, Q - s)$$

После этого естественным образом определим норму на Σ_Q^n :

$$\|m\| = \sum_{i=0}^{n-1} |m_i|$$

Решётками в $\mathbb{R}^n$ станем называть мн-ва L , состоящие из ЛК (*линейных комбинаций*) некоторых n ЛНЗ (*линейно независимых*) векторов.

3 Постановка задачи

Пусть $2 \leq Q \leq q$. Определим отображение $\text{Map} : \Sigma_Q \rightarrow \mathbb{Z}_q$, как:

$$\text{Map}(s) = \left\lfloor \frac{sq}{Q} \right\rfloor$$

В обратную сторону, положим $\text{Demap} : \mathbb{Z}_q \rightarrow \Sigma_Q$, как $\text{Demap}(x)$ — прообраз ближайшего к x по метрике Ли в $\mathbb{Z}_q$ остатка из $\text{Map}(\Sigma_Q)$. Если таких несколько, выберем минимальный из них.

После этого сообщение $m \in \Sigma_Q^n$ станем кодировать и декодировать посимвольно $\text{Map} : \Sigma_Q^n \rightarrow \mathcal{R}_q^n$ и $\text{Demap} : \mathcal{R}_q^n \rightarrow \Sigma_Q^n$.

Теперь можно описать протокол для передачи сообщений:

Алгоритм 1: Генерация ключей

Вход: $q, k \in \mathbb{N}$

$$\mathbf{a} \leftarrow \mathbb{Z}_q$$

$$\mathbf{s}, \mathbf{e} \leftarrow \chi_k$$

$$\mathbf{b} = \mathbf{a}\mathbf{s} + \mathbf{e}$$

Выход: $pk = (\mathbf{b}, \mathbf{a}), sk = \mathbf{s}$

Алгоритм 2: Шифрование

Вход: $m \in \Sigma_Q^n, pk$

$$\mathbf{s}', \mathbf{e}', \mathbf{e}'' \leftarrow \chi_k$$

$$\mathbf{u} = \mathbf{a}\mathbf{s}' + \mathbf{e}'$$

$$\mathbf{v} = \mathbf{b}\mathbf{s}' + \mathbf{e}'' + \text{Map}(m)$$

Выход: $c = (\mathbf{u}, \mathbf{v})$

Алгоритм 3: Разшифровывание

Вход: c, sk

$$\tilde{m} = \text{Demap}(\mathbf{v} - \mathbf{u}\mathbf{s})$$

Выход: $\tilde{m}$

При этом шумом $\mathbf{z}$ назовём $\mathbf{v} - \mathbf{u}\mathbf{s} - \text{Map}(m)$. Имеем:

$$\mathbf{z} = (\mathbf{b}\mathbf{s}' + \mathbf{e}'') - (\mathbf{a}\mathbf{s}' + \mathbf{e}')\mathbf{s} = \mathbf{e}\mathbf{s}' - \mathbf{e}'\mathbf{s} + \mathbf{e}''$$

Сложность взлома этой системы основана на RLWE-problem:

1. Зафиксируем некоторый многочлен $\mathbf{s} \in \mathcal{R}_q^n$.
2. Пусть $\mathbf{a}$ — многочлен из $\mathcal{R}_q^n$ со случайно равномерно независимо в совокупности сгенерированными коэффициентами.
3. Пусть $\mathbf{e}$ — многочлен из $\mathcal{R}_q^n$ со случайно независимо в совокупности сгенерированными из χ_k коэффициентами.
4. По известным $\mathbf{a}$ и $\mathbf{b} = \mathbf{a}\mathbf{s} + \mathbf{e}$ нужно восстановить $\mathbf{s}$.

Данная задача является модификацией LWE-problem:

1. Зафиксируем некоторый вектор $s \in \mathbb{Z}_q^n$.
2. Пусть A — матрица $n \times m$ с элементами, сгенерированными случайно независимо в совокупности равномерно из $\mathbb{Z}_q$.
3. Пусть e — вектор из $\mathbb{Z}_q^n$ со случайно независимо в совокупности сгенерированными из χ_k координатами.
4. По известным A и $b = As + e$ нужно восстановить s .

Известно, что при некоторых требованиях на s и параметры данные задачи являются трудными для решения. Таким образом, велика вероятность и устойчивости к взлому соответствующих им протоколов.

Вообще, сложность задачи измеряется кол-вом итераций алгоритма, её решающим. В случае WCH учитывается худший исход, а при ACH — средняя величина по всем возможным входам.

В частности, P — класс задач, решаемых за полиномиальное время, а NP (*Nondeterministic Polynomial*) — задач, в которых решение за полиномиальное время может быть проверено на правильность.

Кроме того есть мн-во промежуточных классов, с более или менее строгими ограничениями на задачи в них. Тем не менее, до сих пор доподлинно не известно, выполняется ли $P = NP$, хотя большинство учёных и не верят в правдивость столь сильного утверждения.

Возвращаясь к протоколу передачи, заметим, что событие $\tilde{m} \neq m \iff \text{Демар}(\mathbf{z}) \neq 0^n$. Следовательно, можно рассматривать процесс зашифрования, как **RLWE**-канал, который при передаче по нему сообщения $\text{Мар}(m)$ из $\mathcal{R}_q^n$ добавляет к нему шум $\mathbf{z}$ также из $\mathcal{R}_q^n$.

Понятно, что чем выше скорость канала, тем меньше длина сообщения требуется для отправки фиксированного кол-ва информации, и тем быстрее работает передача.

С другой стороны, **DFR** должен оставаться достаточно мал (в этой статье используется порог 10^{-6}). В итоге, полезной задачей является оценить из теоретико-информационных и вычислительных соображений границы на скорость передачи в зависимости от параметров канала.

4 Обзор существующих решений

Многие классические криптографические протоколы становятся полиномиально взламываемыми с изобретением квантового компьютера. В связи с этим, в 2017 году NIST запустил конкурс на роль нового квантово-устойчивого стандарта в шифровании.

Среди лучших вариантов есть алгоритмы, основанные на:

1. Кодов, исправляющих ошибки
2. Целочисленных решётках
3. Хэш-функциях
4. Многочленах от многих переменных
5. Изогениях эллиптических кривых

При этом в силу наличия внутренней структуры многие протоколы на изогениях [17] и многочленах [18] недостаточно устойчивы к взлому.

На хэш-функциях получается построить только подпись без шифрования, а алгоритмы на кодах часто требуют большой длины открытого ключа и, следовательно, мало эффективны.

Криптосистемы же на решётках, в частности, связанные с **LWE**-problem протоколы, обладая достаточной универсальностью и надёжностью, показывают к тому же вполне приемлимую производительность.

Немаловажным является и мощное теоретическое обоснование стойкости. Существует много сложных задач в теории решёток:

1. **SVP** (*Shortest Vector Problem*) — найти ненулевой вектор v из решётки L с минимально возможной длиной $\lambda_1(L) = \|v\|$.
2. SVP_β — найти ненулевой вектор из решётки L с длиной $\leq \beta \cdot \lambda_1(L)$.
3. GapSVP_β — если гарантируется, что $\lambda_1(L) \leq 1$ или $\lambda_1(L) > \beta$, понять, какой вариант для данной L реализуется.

4. **SIVP** (*Shortest Independent Vector Problem*) — найти ЛНЗ систему из n векторов решётки $(v_1, \dots, v_n)$ с минимальным $\lambda_n(L) = \max\{\|v_i\|\}$.

Обычное **SVP** и даже **SVP** $_{\beta}$ при малых β являются NP-сложными [19]. В случае экспоненциальных β задача эффективно решается с помощью LLL-алгоритма [20]. Для полиномиальных β статус не известен, однако задача считается достаточно трудной в худшем случае.

Задача **SIS** $_{\beta}$ (*Short Integer Solution*) описывается так:

1. Пусть A — матрица $n \times m$ с элементами, сгенерированными случайно независимо в совокупности равномерно из $\mathbb{Z}_q$.
2. Надо найти вектор $x \in \mathbb{Z}_m$, такой что $Ax = \vec{0}$ и $0 < \|x\| \leq \beta$.

В 1996 году Ажтай свёл WCH задачи **SVP** и **SIVP** к ACH проблеме **SIS** [8]. Более конкретно, если $\exists$ алгоритм, решающий **SIS** $_{\beta}$ для полиномиального β с вероятностью хотя бы $\frac{1}{2}$, то $\exists$ алгоритм, решающий для некоторых полиномиальных γ также задачи **SVP** $_{\gamma}$ и **SIVP** $_{\gamma}$.

В более поздней работе Регев проделал аналогичное сведение от задач **GapSVP** и **SIVP** к **LWE-problem** [7]. Его рассуждения, можно повторить в терминах **RLWE** и **MLWE**-(*Module LWE*)-problem, и они дают большую уверенность в надёжности решёточных протоколов.

Учитывая всё вышесказанное, полезно изучать свойства криптосистем, основанных на теории решёток вообще и **LWE-problem** в частности, с целью их дальнейшего улучшения и оптимизации.

5 Исследование и построение решения задачи

5.1 Свойства маппинга

Пусть $x \in \mathbb{N}$. Представим xq в виде $Qm + r$ для $0 \leq r < Q$.

$$\left\lfloor \frac{(x+1)q}{Q} \right\rfloor = \left\lfloor \frac{Qm + r + q}{Q} \right\rfloor = m + \left\lfloor \frac{q+r}{Q} \right\rfloor$$

Получаем, что длины $\text{Map}(x+1) - \text{Map}(x)$ отличаются на ≤ 1 , так как

$$\left\lfloor \frac{xq}{Q} \right\rfloor = m; \quad \left\lfloor \frac{q}{Q} \right\rfloor \leq \left\lfloor \frac{q+r}{Q} \right\rfloor \leq \left\lceil \frac{q}{Q} \right\rceil$$

При этом Демар (с точностью до округлений) отправляет в x остатки со 2-ой половины предыдущего $[\text{Map}(x-1), \text{Map}(x)]$ и 1-ой половины следующего $[\text{Map}(x), \text{Map}(x+1)]$ за ним отрезков.

Из доказанного выше свойства про их длины, если зафиксировать символ s , в него точно декодируются все x вида:

$$|x - \text{Map}(s)| < \frac{1}{2} \cdot \left\lfloor \frac{q}{Q} \right\rfloor \iff |x - \text{Map}(s)| < \left\lfloor \frac{q}{2Q} \right\rfloor$$

5.2 Вычисление DFR

Пусть $\mu, \nu \sim \chi_k$ — независимые случайные величины в $\mathbb{Z}_q$. Обозначим распределение их произведения $\mu \cdot \nu$ за ξ_k . Положим $\psi = \xi_k^{\otimes n} * \xi_k^{\otimes n} * \chi_k$.

В статье [12] доказывалось, что это распределение коэффициентов $\mathbf{z}$. Таким образом, полагая их независимыми, получаем $\mathbf{z} \leftarrow \psi$, и

$$b = \left\lfloor \frac{q}{2Q} \right\rfloor; \quad \text{CFR} \leq 1 - \sum_{i=1-b}^{b-1} \psi[i]$$

Математически предположение это неверно, однако широко распространено в силу своей удобства и хорошо себя зарекомендовало на практике.

Утверждение 1. В метрике Хэмминга с исправлением t ошибок DFR можно вычислить по формуле бинома Ньютона, получается

$$\text{DFR} = \sum_{i=t+1}^n C_n^i \cdot \text{CFR}^i \cdot (1 - \text{CFR})^{n-i}$$

Теперь проанализируем, что происходит в метрике Ли. Мы умеем восстанавливать $\tilde{m}$ с $\|\tilde{m} - m\|_L \leq t$. Назовём $x_i = |(\tilde{m} - m)_i|_L$.

Обозначим кроме того $ch : \mathbb{Z}_Q \rightarrow \mathbb{Z}$ функцию вида:

$$ch(x) = \max_{s \in \Sigma_Q} \{ |\text{Demap}(\text{Map}(s) + x) - s| \}$$

Нам подходят вектора $\tilde{m} - m = (x_1, \dots, x_n)$ с $x_1 + \dots + x_n \leq t$. По ψ можно вычислить $\Pr(x_i = k)$ для $0 \leq k < Q$. Более конкретно, пусть

$$\sigma[m] = \sum_{ch(x)=m} \psi[x]$$

Утверждение 2. В метрике Ли с исправлением t ошибок DFR можно вычислить, используя производящие функции, получается

$$P(z) = \sum_{m=0}^{Q-1} \sigma[m] z^m; \quad \text{DFR} = \sum_{i=t+1}^{(Q-1)n} P^n[i]$$

Замечание 1. То же самое работает и для метрики Хэмминга, если взять её $P(z) = (1 - \text{CFR}) + \text{CFR} \cdot z$.

5.3 Границы на скорость кода

Умея считать оценку на DFR при данном t , можно подобрать необходимое кол-во исправлений, чтобы вероятность ошибки была меньше порогового значения.

Примем $V(r)$ — объём шара радиуса r . В Q -ичном алфавите по метрике Хэмминга есть стандартная формула:

$$V_H(r) = \sum_{i=0}^r C_n^i (Q-1)^i;$$

В метрике Ли же всё немного хитрее. Для $0 \leq i < Q$ положим:

$$c_i = |\{s \in \Sigma_Q : |s|_L = i\}| = 2 - I\{i = 0 \vee 2i = Q\}$$

После этого снова воспользуемся производящими функциями:

$$P(z) = \sum_{m=0}^{Q-1} c_m z^m; \quad V_L(r) = \sum_{i=0}^r P^n[i]$$

Замечание 2. Для метрики Хэмминга формула выше получается, если взять $P(z) = 1 + (Q - 1)z$.

Утверждение 3. Для кодового расстояния $d = 2t + 1$ и размерности кода n действуют границы Хэмминга [13] и Варшамова-Гилберта [21, 22] на его размер M :

$$M_H \leq \left\lfloor \frac{Q^n}{V(t)} \right\rfloor; \quad M_{GV} \geq \left\lceil \frac{Q^n}{V(d-1)} \right\rceil$$

После вычисляются границы на скорость оптимального кода:

$$R = \frac{\log_2(M)}{n}$$

6 Описание практической части

Для написания кода использовался язык **Python** и опирающийся на его синтаксис **SageMath**. В качестве криптографических протоколов, основываясь на статье [12], были выбраны **LAC** и **NewHope**.

Зафиксированы следующие наборы параметров:

Протокол	n	q	k
LAC	1024	251	1
NewHope	1024	12289	8

6.1 Вычисление шума

Вероятности исходов $\mathcal{B}(k, \frac{1}{2})$ посчитаны по формулам через модуль `scipy.stats`. После этого распределение χ_k вычислено за время $O(k^2)$ по определению: перебором возможных значений (μ, ν) , вычислением значения $\mu - \nu$ и прибавлением к нему вероятности текущего исхода.

В распределении $\mathcal{B}(k, p)$ присутствуют значения $0 \leq x \leq k$, отсюда в χ_k они будут $-k \leq x \leq k$ — всего $O(k)$ штук. Таким образом, ξ_k также вычисляется по определению за $O(k^2)$ тем же способом.

После этого нам нужно свернуть ξ_k само с собой $2n$ раз, а после этого ещё 1 раз с χ_k для получения распределение шума ψ . Все операции происходят в $\mathbb{Z}_q$, поэтому на каждом шаге распределения принимают не больше q разных значений $0 \leq x < q$.

Сворачивать с собой ξ_k будем, используя алгоритм бинарного возведения в степень и тратя на это $O(\log(n))$ шагов. Во время каждой его итерации, а также в конце при свёртке с χ_k нам нужно находить свёртку распределений со значениями из $\{0, \dots, q-1\}$.

Обозначая их P и Q , значения $P * Q$ ищутся в $\mathbb{Z}_q$ по формуле:

$$(P * Q)[y] = \sum_{x=0}^y P[x]Q[y-x] + \sum_{x=y+1}^{q-1} P[x]Q[q+y-x]$$

Если взять проассоциированные с распределениями многочлены, свёртку

ка выражается через коэффициенты их произведения:

$$(P * Q)[y] = (\hat{P}\hat{Q})[y] + (\hat{P}\hat{Q})[q + y]$$

Но произведение многочленов длины q можно с помощью **NTT** искать за время $O(q \log(q))$. В моём коде для этих целей я использую функцию `fftconvolve` из модуля `scipy.signal`.

В итоге, распределение ψ может быть посчитано с использованием $O(q \log(q) \log(n) + k^2)$ времени и $O(q \log(n))$ памяти, чего вполне хватает для большинства используемых сейчас криптосистем.

6.2 Вероятности ошибок

Дальше, для быстрого вычисления Демар предподсчитаем его значения. Будем перебирать символ $x \in \mathbb{Z}_q$ и смотреть, к образу какого из $s \in \Sigma_Q$ он ближе — асимптотика $O(qQ)$.

Теперь Мар и Демар считаются за $O(1)$, и можно по определению вычислить значения ch — также за $O(qQ)$.

Зная ψ , по ним за $O(q)$ восстанавливается распределение σ . Наконец, для оценки **DFR** у нас снова возникает задача посчитать P^n , но уже в $\mathbb{R}[x]$.

Посмотрим на устройство алгоритма бинарного возведения в степень. Так как $\deg(P) = Q - 1$ и, следовательно, $\deg(P^m) \leq (Q - 1)m$, можно дополнять P^m нулями до длины Qm .

В процессе вычисления мы ≤ 2 раза перемножим многочлены длины $\frac{1}{2}Qn$, ≤ 2 раза — длины $\frac{1}{4}Qn$ и т.д. С **NTT** суммарно получается:

$$Time \leq \sum_{i=1}^B \frac{1}{2^i} Qn \cdot \log \left(\frac{1}{2^i} Qn \right)$$

Огрубляя и оценивая сумму геометрической прогрессии, выводим:

$$Time \leq Qn \log(Qn)$$

Имеем асимптотику $O(Q(n \log(Qn) + q))$ времени и $O(q + Qn)$ памяти.

6.3 Подсчёт t и R

Утверждение 4. *Значение DFR при фиксированном n убывает при возрастании t , из-за этого первый момент выполнения $\text{DFR} < 10^{-6}$ можно искать с помощью алгоритма бинарного поиска.*

Вычисление DFR происходит суммированием за $O(Qn)$, значение t ограничено тем же числом — асимптотика $O(Qn \log(Qn))$.

Дальше по данным t и n нужно вычислить M . В случае Хэмминга я применяю для этого методы `gilbert_lower_bound` и `hamming_upper_bound` из библиотеки `sage.codes.bounds`.

В метрике Ли я явно строю P , как элемент `PolynomialRing(ZZ)` из `SageMath`, и после встроенным методом считаю P^n , откуда узнаю объёмы шаров и границы на M соответственно.

Данный шаг снова работает за асимптотику $O(Qn \log(Qn))$. Наконец, чтобы узнать R , я рассматриваю M , как элемент `RealField(100)` из `SageMath`, и встроенной функцией нахожу его логарифм.

7 Численные результаты

Вот результаты для **LAC** и **NewHope**. Параметры $[n, q, k]$ были взяты $[1024, 251, 1]$ и $[1024, 12289, 8]$ соответственно. Целевой DFR положим 10^{-6} .

Для **LAC** имеет смысл рассматривать размер алфавита Q от 2 до 6 символов включительно, подбираем на компьютере параметры:

Таблица 1 – Хэмминг, Утв 1 и 4

Q	t	err
2	21	0.0058
3	109	0.0668
4	235	0.1709
5	345	0.2695
6	448	0.3651

Таблица 2 – Ли, Утв 2 и 4

Q	t	err1	err2
2	21	0.0058	0
3	109	0.0668	0
4	235	0.1709	3.64e-05
5	346	0.2686	0.0009
6	456	0.3594	0.0058

Последний столбец отвечает за вероятность ошибки в каждом символе (если рассматривать их независимо). В метрике Ли это вероятности отличия от исходного символа на расстояние 1 и 2 соответственно.

Для **NewHope** делаем то же самое, только рассматривая другие Q :

Таблица 3 – Хэмминг, Ли, Утв 1,2 и 4

Q	t	err
2	0	≈ 0
3	0	≈ 0
4	0	≈ 0
5	0	1.52e-11
6	1	1.71e-08
7	1	1.32e-06
8	3	2.28e-05
9	5	1.66e-04

Для маленьких размеров алфавита вероятность двойной ошибки пренебрежимо мала, так что результаты одинаковы.

Таблица 4 – Хэмминг, Утв 1 и 4

Q	t	err
10	7	0.0007
12	18	0.0047
14	38	0.0155
16	65	0.0339
18	100	0.0596
20	138	0.0899
23	199	0.1402
26	259	0.1923
30	333	0.2585
34	398	0.3186
38	455	0.3722

Таблица 5 – Ли, Утв 2 и 4

Q	t	err1	err2
10	7	0.0007	≈ 0
12	18	0.0047	≈ 0
14	38	0.0155	≈ 0
16	65	0.0339	2.32e-10
18	100	0.0596	1.74e-08
20	138	0.0899	3.87e-07
23	199	0.1402	1.00e-05
26	259	0.1922	0.0001
30	334	0.2578	0.0007
34	402	0.3158	0.0028
38	466	0.3647	0.0075

Тем не менее, при больших значениях Q разница проявляется. Далее по известным n и d можно посчитать границы на скорость R .

Таблица 6 – ЛАС, Хэмминг, Утв 3

Q	R_{GV}	R_H
2	0.7569	0.8591
3	0.6298	0.9939
4	0.2821	0.8639
5	0.0674	0.7312
6	0.0010	0.5854

Таблица 7 – ЛАС, Ли, Утв 3

Q	R_{GV}	R_H
2	0.7569	0.8591
3	0.6298	0.9939
4	0.4507	0.9767
5	0.3495	0.9804
6	0.3072	0.9879

Видно, что при маленьких Q и t разница между метриками Хэмминга и Ли небольшая или вообще отсутствует, при увеличении же этих параметров Хэмминг начинает резко убывать, а Ли почти не уменьшается (за счёт гораздо меньшего объёма шаров, возникающих вокруг кодовых слов).

Таблица 8 – NewHope, Хэмминг, Утв 3

Q	R_{GV}	R_H
2	1.0000	1.0000
3	1.5850	1.5850
4	2.0000	2.0000
5	2.3219	2.3219
6	2.5619	2.5729
7	2.7838	2.7951
8	2.9342	2.9650
9	3.0643	3.1132
10	3.1775	3.2439
12	3.2475	3.3999
14	3.1555	3.4450
16	2.9596	3.4151
18	2.6640	3.3137
20	2.3412	3.1837
23	1.8315	2.9513
26	1.3565	2.7150
30	0.8186	2.4222
34	0.4060	2.1680
38	0.1187	1.9473

Таблица 9 – NewHope, Ли, Утв 3

Q	R_{GV}	R_H
2	1.0000	1.0000
3	1.5850	1.5850
4	2.0000	2.0000
5	2.3219	2.3219
6	2.5645	2.5742
7	2.7868	2.7966
8	2.9448	2.9703
9	3.0838	3.1230
10	3.2070	3.2587
12	3.3331	3.4429
14	3.3518	3.5442
16	3.3163	3.5966
18	3.2367	3.6080
20	3.1565	3.6074
23	3.0427	3.5936
26	2.9581	3.5841
30	2.8863	3.5841
34	2.8495	3.5972
38	2.8290	3.6139

Также интересно, что вероятность возникновения отличия на ≥ 2 в метрике Ли перестаёт быть пренебрежительно мала лишь к моменту, когда вероятность отличия на 1 становится огромна (порядка $\frac{1}{4}$ и выше).

Таким образом, пока кол-во исправляемых ошибок остаётся не слишком большим, можно считать их все переходами из символа в соседний.

8 Заключение

В статье рассматривались криптосистемы **LAC 256** и **NewHope 1024**. Процесс шифрования в них был рассмотрен, как передача данных по зашумлённому **RLWE**-каналу.

Были исследованы свойства этого канала, в предположении независимости компонент шума оценена вероятность ошибочной декодировки сообщения, проанализировано кол-во ошибок, которое необходимо уметь исправлять для получения заданного **DFR** в метриках Хэмминга и Ли.

Используя границы Варшамова-Гилберта и Хэмминга, были получены оценки на скорость оптимальной передачи данных по каналу при фиксированной метрике и размере алфавита Q .

Получено, что для **LAC** оптимальными Q являются 2 и 3. При этом метрики Хэмминга и Ли с такими размерами алфавита эквивалентны.

Таблица 10 – LAC, Хэмминг, Табл 6

Q	R_{GV}	R_H
2	0.7569	0.8591
3	0.6298	0.9939

Таблица 11 – LAC, Ли, Табл 7

Q	R_{GV}	R_H
2	0.7569	0.8591
3	0.6298	0.9939

Для **NewHope** ситуация интереснее, рассмотрим несколько Q :

Таблица 12 – NewHope, Хэмминг, Табл 8

Q	R_{GV}	R_H
8	2.9342	2.9650
14	3.1555	3.4450
34	0.4060	2.1680

Таблица 13 – NewHope, Ли, Табл 9

Q	R_{GV}	R_H
8	2.9448	2.9703
14	3.3518	3.5442
34	2.8495	3.5972

При маленьких Q ошибок почти не происходит, исправлять их легко, и скорость получается близкой к максимально возможной $\log_2(Q)$.

Где-то в районе $Q = 14$ выигрыш за счёт увеличения размера алфавита уравнивается возрастанием вероятности ошибки, и скорость передачи достигает своего максимума.

Как и следовало ожидать, более аккуратный анализ ошибок в метрике Ли даёт несколько лучшие показатели в этом случае.

При дальнейшем увеличении Q скорость начинает падать. И если в метрике Хэмминга уменьшение ярко выражено, в метрике Ли оно не велико, а верхняя граница вообще остаётся почти константной.

Это происходит, так как практически все ошибки при передаче меняют символ на соседний. Соответственно, метрика Ли учитывает оное при анализе, чего Хэмминг сделать не в состоянии.

Список литературы

- [1] *Rivest, R. L.* A method for obtaining digital signatures and public-key cryptosystems / R. L. Rivest, A. Shamir, L. Adleman // *Commun. ACM*. — 1978. — Vol. 21, no. 2. — P. 120–126.
- [2] *Dolmatov, Vasily.* GOST R 34.10-2012: Digital Signature Algorithm. — RFC 7091. — 2013.
- [3] *Smyshlyaev, Stanislav V.* Guidelines on the Cryptographic Algorithms to Accompany the Usage of Standards GOST R 34.10-2012 and GOST R 34.11-2012. — RFC 7836. — 2016.
- [4] *Shor, Peter W.* Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer / Peter W. Shor // *SIAM Journal on Computing*. — 1997. — Vol. 26, no. 5. — P. 1484–1509.
- [5] *Proos, John.* Shor’s discrete logarithm quantum algorithm for elliptic curves / John Proos, Christof Zalka // *Quantum Info. Comput.* — 2003. — Vol. 3, no. 4. — P. 317–344.
- [6] *Yesina, Maryna.* Status report on the third round of the NIST post-quantum cryptography standardization process / Maryna Yesina, Ye.V. Ostrianska, I.D. Gorbenko // *Radiotekhnika*. — 2022. — Pp. 75–86.
- [7] *Regev, Oded.* On lattices, learning with errors, random linear codes, and cryptography / Oded Regev // *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing*. — STOC ’05. — New York, NY, USA: Association for Computing Machinery, 2005. — P. 84–93.
- [8] *Ajtai, M.* Generating hard instances of lattice problems (extended abstract) / M. Ajtai // *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*. — STOC ’96. — New York, NY, USA: Association for Computing Machinery, 1996. — P. 99–108.

- [9] *Lyubashevsky, Vadim*. On Ideal Lattices and Learning with Errors over Rings / Vadim Lyubashevsky, Chris Peikert, Oded Regev // *J. ACM*. — 2013. — Vol. 60, no. 6. — 35 pp.
- [10] *Liang, Zhichuang*. Number Theoretic Transform and Its Applications in Lattice-based Cryptosystems: A Survey. — 2022.
- [11] *Li, Yang*. A Tutorial Introduction to Lattice-based Cryptography and Homomorphic Encryption / Yang Li, Kee Siong Ng, Michael Purcell // *ArXiv*. — 2022. — Vol. abs/2208.08125.
- [12] *Maringer, Georg*. Higher Rates and Information-Theoretic Analysis for the RLWE Channel / Georg Maringer, Sven Puchinger, Antonia Wachter-Zeh // 2020 IEEE Information Theory Workshop (ITW). — 2021. — Pp. 1–5.
- [13] *Hamming, R. W.* Error Detecting and Error Correcting Codes / R. W. Hamming // *Bell System Technical Journal*. — 1950. — Vol. 29, no. 2. — Pp. 147–160.
- [14] *Lee, C. Y.* Some properties of nonbinary error-correcting codes / C. Y. Lee // *IRE Trans. Inf. Theory*. — 1958. — Vol. 4. — Pp. 77–82.
- [15] LAC: Practical Ring-LWE Based Public-Key Encryption with Byte-Level Modulus / Xianhui Lu, Yamin Liu, Zhenfei Zhang et al. // *IACR Cryptol. ePrint Arch*. — 2018. — Vol. 2018. — P. 1009.
- [16] Post-quantum key exchange: a new hope / Erdem Alkim, Léo Ducas, Thomas Pöppelmann, Peter Schwabe // Proceedings of the 25th USENIX Conference on Security Symposium. — SEC'16. — USA: USENIX Association, 2016. — P. 327–343.
- [17] *Castryck, Wouter*. An Efficient Key Recovery Attack on SIDH / Wouter Castryck, Thomas Decru. — Berlin, Heidelberg: Springer-Verlag, 2023. — P. 423–447.

- [18] *Beullens, Ward*. Breaking Rainbow Takes a Weekend on a Laptop / Ward Beullens. — 2022. — Pp. 464–479.
- [19] *Hair, Isaac M*. SVP_p is Deterministically NP-Hard for all $p > 2$, Even to Approximate Within a Factor of $2^{\log^{1-\epsilon} n}$. — Cryptology ePrint Archive, Paper 2025/2181. — 2025.
- [20] *Lenstra H.W. jr., Lenstra A.K. Lovász L*. Factoring Polynomials with Rational Coefficients. / Lenstra A.K. Lovász L. Lenstra, H.W. jr. // *Mathematische Annalen*. — 1982. — Vol. 261. — Pp. 515–534.
- [21] *Gilbert, E. N*. A comparison of signalling alphabets / E. N. Gilbert // *The Bell System Technical Journal*. — 1952. — Vol. 31, no. 3. — Pp. 504–522.
- [22] *Varshamov, R. R*. Estimate of the number of signals in error correcting codes / R. R. Varshamov // *Doklady Akad. Nauk, S.S.S.R.* — 1957. — Vol. 117. — Pp. 739–741.
- [23] *Savvateev, M. A*. Bachelor thesis materials. — 2026. https://github.com/Dart-Xeyter/Graduate_Thesis.

Приложение

Целиком реализованный код можно найти на моём [Github](#) по ссылке [23]. Здесь проиллюстрирую только фрагменты основных этапов.

Листинг 1 – Подсчёт распределения шума

```
def comp_distrib(a, b, op):
    inds_a = [(q, x) for q, x in enumerate(a) if x != 0]
    inds_b = [(q, x) for q, x in enumerate(b) if x != 0]
    res = [0]*len(a)
    for q1, x in inds_a:
        for q2, y in inds_b:
            res[op(q1, q2)] += x*y
    return res
```

```
def chi_distrib(q, k):
    Bin = [binom.pmf(q1, k, 0.5) for q1 in range(k+1)]
    Bin += [0]*(q-k-1)
    return comp_distrib(Bin, Bin, minus)
```

```
def noise_distrib(chi, n):
    ksi = comp_distrib(chi, chi, mul)
    ksi_pow = pow_distrib(ksi, 2*n)
    return comp_distrib(ksi_pow, chi, plus)
```

Листинг 2 – Вычисление DFR

```
def err_prob_Li(noise, n, q, Q):
    ch = max_changes(Q, q)
    err = [0]*Q
    for q1 in range(q):
        err[ch[q1]] += noise[q1]
    errors.append(err[:3])
    errs = fft_pow(err, n)
    return err, errs
```

```
def get_DFR_Li(err, n, t):
    return sum(err[t+1:])
```

Листинг 3 – Нахождение кол-ва исправляемых ошибок

```
def get_t(err, N, get_DFR):
    ERR = ["10**-6"]
    ans = []
    for dfr in ERR:
        dfr = eval(dfr)
        l, r = -1, N+1
        while r-l > 1:
            t = (l+r)//2
            DFR = get_DFR(err, N, t)
            if DFR < dfr:
                r = t
            else:
                l = t
        ans.append(r)
    return ERR, ans
```

Листинг 4 – Получение границ на скорость передачи

```
def get_R_Li(n, Q, t):
    RF = RealField(100)
    R.<x> = PolynomialRing(ZZ)
    coefs = [0]*Q
    for q1 in range(Q):
        coefs[min(q1, Q-q1)] += 1
    nums = (R(coefs)^n).list()
    V_down, V_up = sum(nums[:2*t+1]), sum(nums[t+1:])
    M_down, M_up = Q**n//V_down, Q**n//V_up
    R_down, R_up = log(RF(M_down), 2)/n, log(RF(M_up), 2)/n
    return float(R_down), float(R_up)
```